IBM BOB HACKATHON · TRACK 2 — MODERNIZE WHAT MATTERS

# ParityProof

Modernize COBOL to Java and **prove the new code behaves exactly like the original** — a copybook-aware runner diffs the COBOL reference and the Java twin field-by-field, on identical inputs.

### ✔ It runs

Two original COBOL batch programs compiled under GnuCOBOL; Java 21 twins; edge-case input generators. One command: `bash run.sh`.

### ✔ It proves

**100% field parity (64/64 fields)**, and it catches real divergence (injected rounding bug → 83% on ACCOUNT-INT, exact fields pinpointed).

### ✔ It is central to IBM Bob

Bob explains the rules (Ask mode), plans a change, and edits the Java (Agent mode); a parity hook re-runs the proof after Bob's edits (one real firing is logged). Sessions and reviews are committed.

**Team Mavericks:** Abdul Qaadir (Java lead · Bob driver) · Satish (Cloud · CI) · Karthik (Business analysis · story)

THE PROBLEM

# Banks want to modernize. They're afraid to.

### 80%+ of transactions

Run on IBM i / IBM Z, much of it in COBOL batch that has run untouched for decades.

### Senior COBOL developers are retiring

The pattern is a knowledge cliff — the people who understand the business rules are leaving.

### Modernization risk is the blocker

"Tests pass" on generated code means nothing until you've shown the new output matches the old.

**The question every bank asks first is not "did you generate code?" — it's "is it the same?"**

WHY THIS WINS

# Many teams work with Bob. Few prove the output is correct.

The first thing any bank wants is a guarantee the rewrite is behaviorally identical. We built the mechanism that delivers it.

### Most teams

Show generated Java → judge sees code, must trust it.

### ParityProof

Adds **a running proof the Java matches the COBOL byte-for-byte**, with Bob in the loop (Ask / Agent modes).

### Judging edge

Correctness + explainability is the defensible innovation, not the generation itself.

**100%**
field parity — 64/64 (2 programs)

**83%**
→ FAIL on ACCOUNT-INT when a rounding slip is injected (20/24) — fields pinpointed

**14**
edge-case records: 6 interest + 8 transaction posting

WHAT WE BUILT

# ParityProof pipeline

- **COBOL reference runtime** — GnuCOBOL (verified 3.2.0) runs two original batch programs: ACCOUNT-INT and TXN-POST (signed amounts, COMP-3). A Dockerfile exists but is not yet built.
- **Business-rule catalog** — drafted with Bob and Claude Code, line citations verified; Karthik's business review is pending.
- **Java 21 twins** — plain single-file programs; BigDecimal arithmetic; copybook layout → fixed-width output.
- **Edge-case input generators** — half-cent rounding, zero and tiny balances, negative balances, sign handling, and field-limit overflow (silent COBOL truncation, reproduced).
- **Parity runner** — diffs every field of every record, per program, by the copybook layout → result.json; exits non-zero on any mismatch.
- **Gate + dashboard** — `run.sh` fails unless parity is 100%; a GitHub Actions workflow is written (not yet run on GitHub); a static dashboard is live on GitHub Pages (bpmforbusiness.github.io/parityproof).

Every output keeps the original fixed-width format so the diff is exact.

LIVE PROOF

# Parity catches real bugs

Interest batch, acct 1: COBOL `00000043208` (432.08). Java with rounding changed to DOWN (injected): `00000043207` (432.07).

**CORRECT**
HALF_UP → 100% pass, 64/64 fields

**INJECTED BUG**
DOWN → ACCOUNT-INT 20/24 FAIL, INTEREST + NEW-BAL flagged (a deliberate mutation, not a Bob change)

Every run is saved under **evidence/runs/<timestamp>/** (inputs, both outputs, result.json): a real PASS and a real injected FAIL are committed, and the runner also fails cleanly when a COBOL or Java output is missing.

HOW IBM BOB IS CENTRAL

# Bob in the loop at every step

### Ask — extract rules (skill used)

Bob explained TXN-POST as business rules with source-line citations — all citations verified. Review caught its mistakes and kept its real finding (batch total repeated on every record; no per-account grouping).

### Plan a change (in Ask mode)

Bob planned a comment-only change to the Java twin, citing the COBOL lines, with a parity run as the acceptance gate.

### Agent — edit the Java

Bob made a comment-only edit with `apply_diff`; the **parity hook fired automatically** and logged **64/64 PASS** (evidence/hooks/). A FAIL firing is not yet captured.

### Governance

Rules file (COMP-3 → BigDecimal, COBOL truncation, cite source lines) + cobol-explainer skill + parity hook script.

### Adoption evidence

Bob replies, our reviews of them and six screenshots (skill used, hooks active, edit) are committed under **bob/evidence-files/**. The Bobalytics export is still to be added.

TECH STACK

# Built with

GnuCOBOL 3.2 | Java 21 / BigDecimal | Single-file Java, no build step | Bash pipeline (run.sh) | Static HTML dashboard | GitHub Actions (written) | Dockerfile (not yet built) | IBM Bob (Ask / Agent modes)

## Correctness infrastructure

Copybook-driven field diff; BigDecimal arithmetic; signed and COMP-3 fields; fixed-width I/O so diffs are exact.

## Repeatability

One command (`bash run.sh`) reproduces the whole run and keeps timestamped evidence. Docker and CI definitions are provided but not yet exercised.

## Evidence machine-readable

result.json drives the static dashboard: overall and per-program parity, per-field rates, mismatch drill-down.

FUTURE ROADMAP & LEARNINGS

# Next steps

- **Business review** — Karthik signs off the rule catalogs; decide whether the silent overflow truncation (preserved for parity) should become a guard in production.
- **True packed-decimal files** — our files are DISPLAY text; binary COMP-3 record I/O is the next hard case.
- **Ship the gate** — run the CI workflow on GitHub (the dashboard is already hosted).
- **More programs** — statements and other batch jobs; AWS CardDemo source once its license is confirmed.

## Key learning

Parity-as-a-test-harness changes how a migration is trusted: it turns an article of faith ("Bob's code works") into a measurable, verifiable guarantee — which is exactly the evidence a bank asks for before signing off. It also surfaced hidden COBOL behavior (silent overflow truncation) that a naive rewrite would have "fixed" and broken parity.